Name: Mandar Uttam Ghadigaonkar
Class: Msc.IT Part 1
Exam Seat no: MSC2514103
Roll No: 08
Semester: 2
Subject: Big Data
Academic Year: 2025-26

UNIVERSITY OF MUMBAI
SHETH T.J. EDUCATION SOCIETY'S,
SHETH N.K.T.T. COLLEGE OF COMMERCE
& SHETH J.T.T. COLLEGE OF ARTS, THANE (W)

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that **Mr. Mandar Uttam Ghadigaonkar** of MSc.IT Part 1 -Sem 2. have completed the practical work in the subject of **Big Data** during the academic year 2025-26 under the guidance of **Mrs**. **Sneha Parekh** being the partial requirement for the fulfillment of the curriculum of Degree of Master of Science in Information Technology, University of Mumbai.

Place: Thane
Date:

Sign of Subject in Charge	**Sign of External Examiner**	**Sign of Co-Ordinator**

INDEX

Sr.No	Practical Name	Date	Sign
1	Install, configure, and run Hadoop and HDFS		
2	Exploring Hadoop Distributed File System (HDFS)		
3	Implement an application that stores big data in HBase/MongoDB/Pig using Hadoop/R		
4	Implement a program in Pig		
5	Illustrate the working of Jaql		
6	Implement Decision Tree classification technique		
7	Implement SVM Classification technique		
8	Regression Model: Import data from web storage, perform Logistic Regression to find relations between variables affecting student admission based on GRE score, GPA, and rank. Check model fit (foreign) and requirement (Mass).		
9	Multiple Regression Model: Apply multiple regression if data has a continuous independent variable. Use the same dataset.		
10	Classification Model: a) Install relevant packages b) choose a classifier, and c) evaluate its performance.		
11	Clustering Model: a) Apply clustering algorithms for unsupervised classification and b) Plot cluster data using R visualizations		

Big Data Analytics Practical
Practical No: 1

Aim: Install, configure and run Hadoop and hdfs

Description: Hadoop is an open-source framework for distributed storage and processing of large datasets using the MapReduce programming model.

Step 1: Install Java JDK 1.8

Download Java JDK from the official link:
https://www.oracle.com/java/technologies/javase-jdk8-downloads.html

Extract and install Java in C:\Java

Open cmd and type:

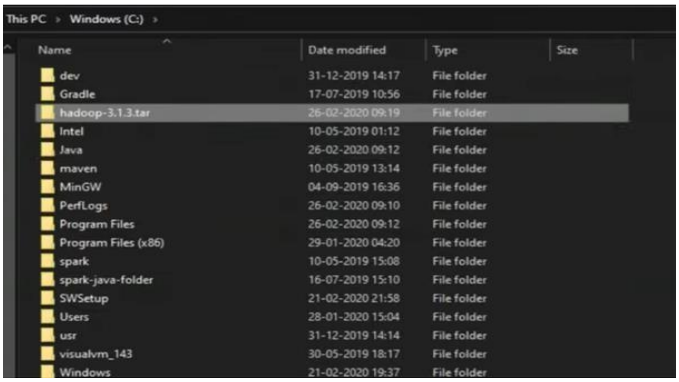
```
C:\>javac -version
javac 20
```

to verify Java installation.

2. Download Hadoop

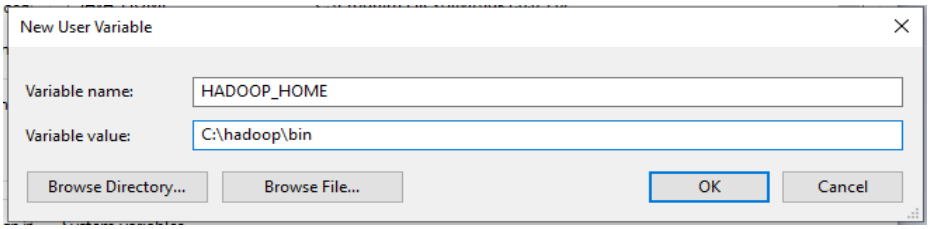
- **Download Hadoop from:**
  https://www.apache.org/dyn/closer.cgi/hadoop/common/hadoop-3.3.0/hadoop-3.3.0.tar.gz
- **Right-click the .tar.gz file → Show more options → 7-zip → **Extract to C:\Hadoop-3.3.0***

Step 2: Extract Hadoop at C:\Hadoop



Step 3: Setting up the HADOOP_HOME variable

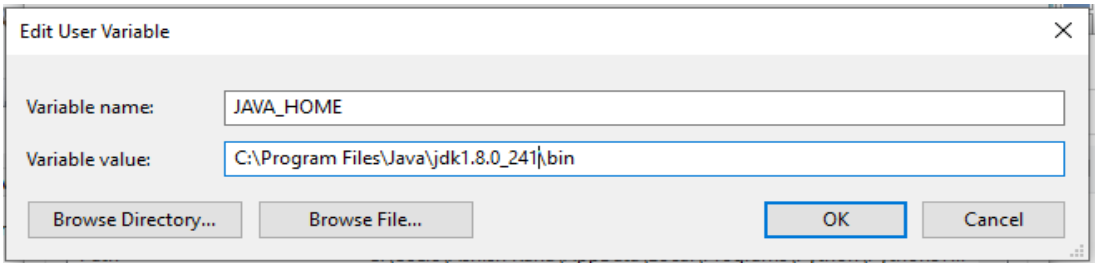
Use windows environment variable setting for Hadoop Path setting.



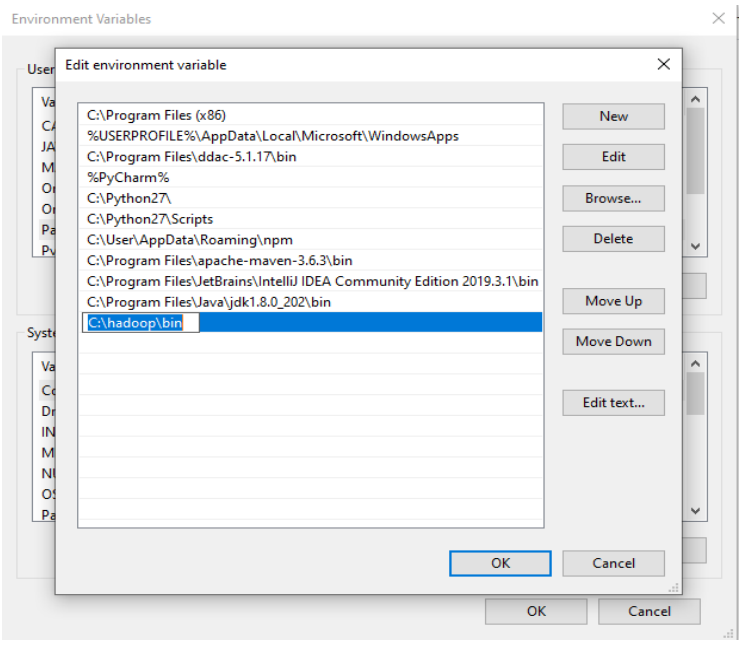
/

Step 4: Set JAVA_HOME variable

Use windows environment variable setting for Hadoop Path setting.



Step 5: Set Hadoop and Java bin directory path



Step 6: Hadoop Configuration :

For Hadoop Configuration we need to modify Six files that are listed below-

1. **Core-site.xml**
2. **Mapred-site.xml**
3. **Hdfs-site.xml**
4. **Yarn-site.xml**
5. **Hadoop-env.cmd**

Create two folders datanode and

6. namenode Step 6.1: Core-site.xml

configuration

```
<configuration>
  <property>
      <name>fs.defaultFS</name>
      <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

Step 6.2: Mapred-site.xml configuration

```
<configuration>
  <property>
      <name>mapreduce.framework.name</name>
      <value>yarn</value>
  </property>
</configuration>
```

Step 6.3: Hdfs-site.xml configuration

```
<configuration>
   <property>
      <name>dfs.replication</name>
      <value>1</value>
   </property>
   <property>
      <name>dfs.namenode.name.dir</name>
      <value>C:\hadoop-2.8.0\data\namenode</value>
   </property>
   <property>
```

```
    <name>dfs.datanode.data.dir</name>
    <value>C:\hadoop-2.8.0\data\datanode</value>
  </property>
</configuration>
```

Step 6.4: Yarn-site.xml configuration

```
<configuration>
  <property>
     <name>yarn.nodemanager.aux-services</name>
     <value>mapreduce_shuffle</value>
  </property>
  <property>
       <name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>
   <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  </property>
</configuration>
```

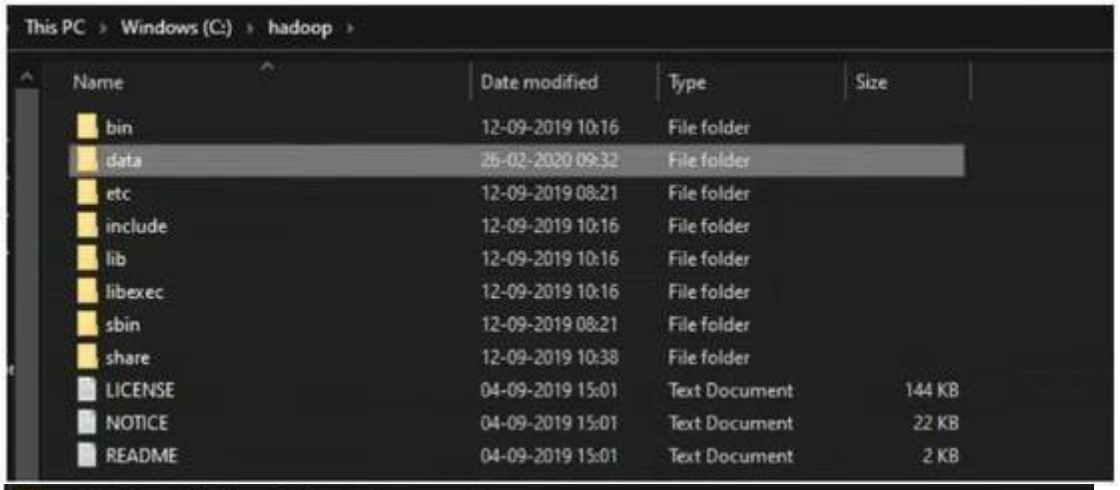
Step 6.5: Hadoop-env.cmd configuration

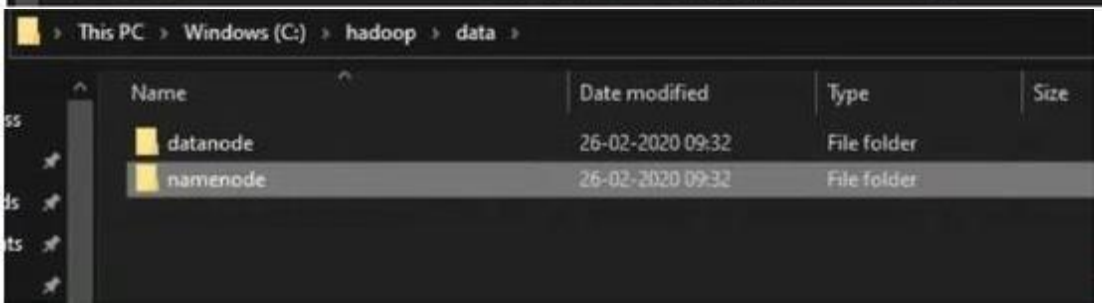
Set "JAVA_HOME=C:\Java" (On C:\java this is path to file jdk.18.0)

```
16
17  @rem Set Hadoop-specific environment variables here.
18
19  @rem The only required environment variable is JAVA_HOME.  All others are
20  @rem optional.  When running a distributed configuration it is best to
21  @rem set JAVA_HOME in this file, so that it is correctly defined on
22  @rem remote nodes.
23
24  @rem The java implementation to use.  Required.
25  set JAVA_HOME=%JAVA_HOME%
```

Step 6.6: Create datanode and namenode folders

1. **Create folder "data" under "C:\Hadoop-2.8.0"**
2. **Create folder "datanode" under "C:\Hadoop-2.8.0\data"**
3. **Create folder "namenode" under "C:\Hadoop-2.8.0\data"**

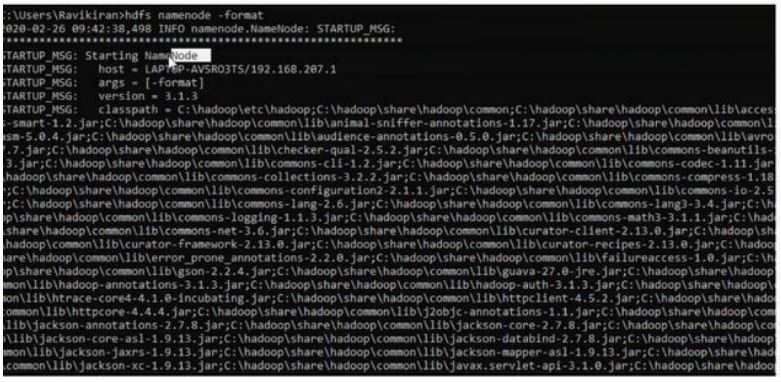




Step 7: Format the namenode folder

Open command window (cmd) and typing command "hdfs namenode –format"

This shows we have successfully install Hadoop



Step 8: Testing the setup

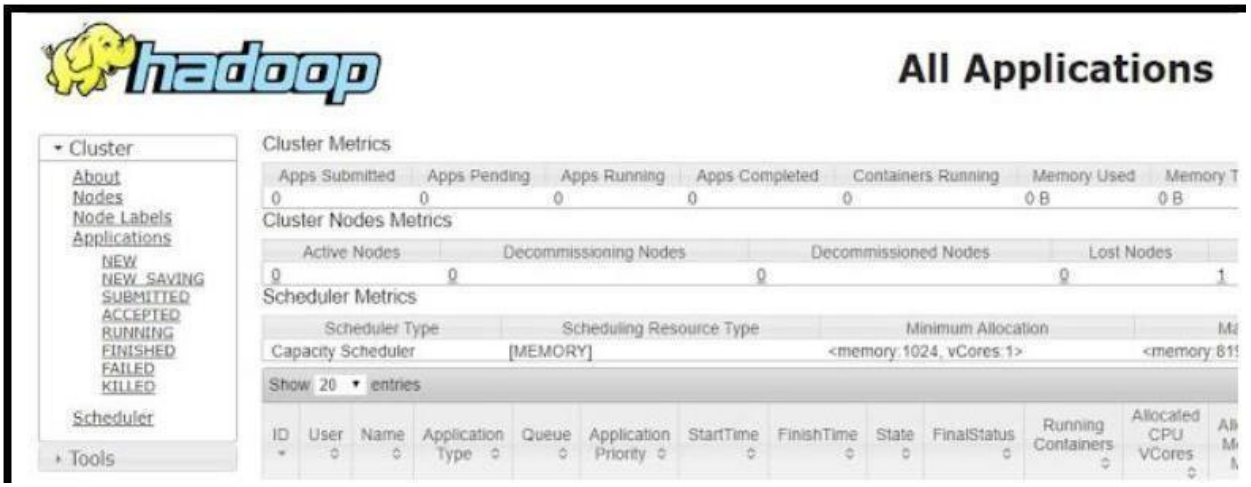
Open command window (cmd) and typing command "start-all.cmd"

```
C:\hadoop>cd sbin

C:\hadoop\sbin>start-all.cmd
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
```

Step 8: Testing the setup:

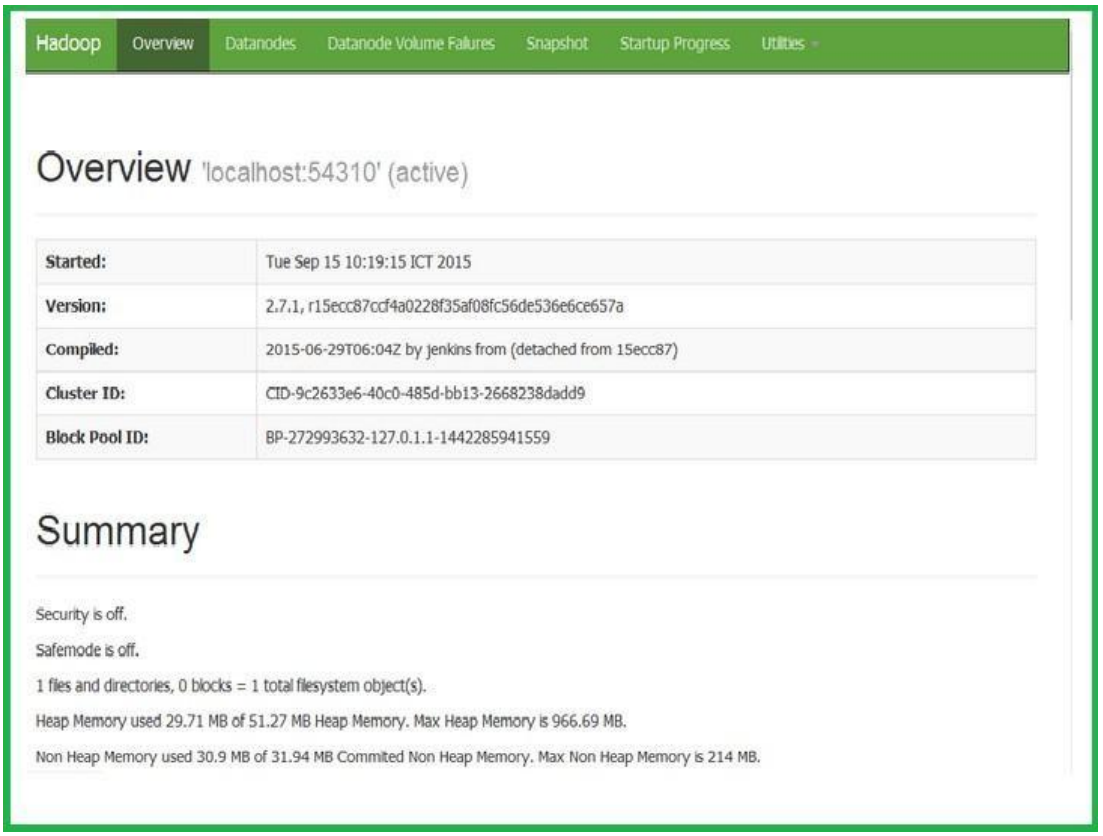
Ensure that namenode, datanode, and Resource manager are running

Step 9: Open: http://localhost:8088



Step 10:

Open: http://localhost:50070



Practical No: 2

Aim: Exploring Hadoop Distributed File System (HDFS)

Description: Hadoop Distributed File System (HDFS) is a file system that manages large data sets that can run on commodity hardware. HDFS is the most popular data storage system for Hadoop and can be used to scale a single Apache Hadoop cluster to hundreds and even thousands of nodes

Hadoop Distributed File System (HDFS) Based on the Google File System [12], the Hadoop Distributed File System (HDFS) is a file system that provides the capability to distribute data across a cluster to take advantage of the parallel processing of MapReduce. HDFS is not an alternative to common file systems, such as ext3, ext4, and XFS. In fact, HDFS depends on each disk drive's file system to manage the data being stored to the drive media. The Hadoop Wiki [14] provides more details on disk configuration options and considerations. For a given file, HDFS breaks the file, say, into 64 MB blocks and stores the blocks across the cluster. So, if a file size is 300 MB, the file is stored in five blocks: four 64 MB blocks and one 44 MB block. If a file size is smaller than 64 MB, the block is assigned the size of the file. Whenever possible, HDFS attempts to store the blocks for a file on different machines so the map step can operate on each block of a file in parallel. Also, by default, HDFS creates three copies of each block across the cluster to provide the necessary redundancy in case of a failure. If a machine fails, HDFS replicates an accessible copy of the relevant data blocks to another available machine. HDFS is also rack aware, which means that it distributes the blocks across several equipment racks to prevent an entire rack failure from causing a data unavailable event. Additionally, the three copies of each block allow Hadoop some flexibility in determining which machine to use for the map step on a particular block. For example, an idle or underutilized machine that contains a data block to be processed can be scheduled to process that data block. To manage the data access, HDFS utilizes three Java daemons (background processes): NameNode, DataNode, and Secondary NameNode. Running on a single machine, the NameNode daemon determines and tracks where the various blocks of a data file are stored. The DataNode daemon manages the data stored on each machine. If a client application wants to access a particular file stored in HDFS, the application contacts the NameNode, and the NameNode provides the application with the locations of the various blocks for that file. The application then communicates with the appropriate DataNodes to access the file. Each DataNode periodically builds a report about the blocks stored on the DataNode and sends the report to the NameNode. If one or more blocks are not accessible on a DataNode, the NameNode ensures that an accessible copy of an inaccessible data block is replicated to another machine. For performance reasons, the NameNode resides in a machine's memory. Because the NameNode is critical to the operation of HDFS, any unavailability or corruption of the NameNode results in a data unavailability event on the cluster. Thus, the NameNode is viewed as a single point of failure in the Hadoop environment [15]. To minimize the chance of a NameNode failure and to improve performance, the NameNode is typically run on a dedicated machine. A third daemon, the Secondary NameNode, provides the capability to perform some of the NameNode tasks to reduce the load on the NameNode. Such tasks include updating the file system image with the contents of the file system edit logs. It is important to note that the Secondary NameNode is not a backup or redundant NameNode. In the event of a NameNode outage, the NameNode must be restarted and initialized with the last file system image file and the contents of the edits logs. The latest versions of Hadoop provide an HDFS High Availability (HA) feature. This feature enables the use of two NameNodes: one in an active state, and the other in a standby state. If an active NameNode fails, the standby NameNode takes over. When using the HDFS HA feature, a Secondary NameNode is unnecessary [16]. Figure 10.2 illustrates a Hadoop cluster with ten machines and the storage of one large file requiring three HDFS data blocks. Furthermore, this file is stored using triple replication. The machines running the NameNode and the Secondary NameNode are considered master nodes. Because the DataNodes take their instructions from the master nodes, the machines running the DataNodes are referred to as worker nodes.

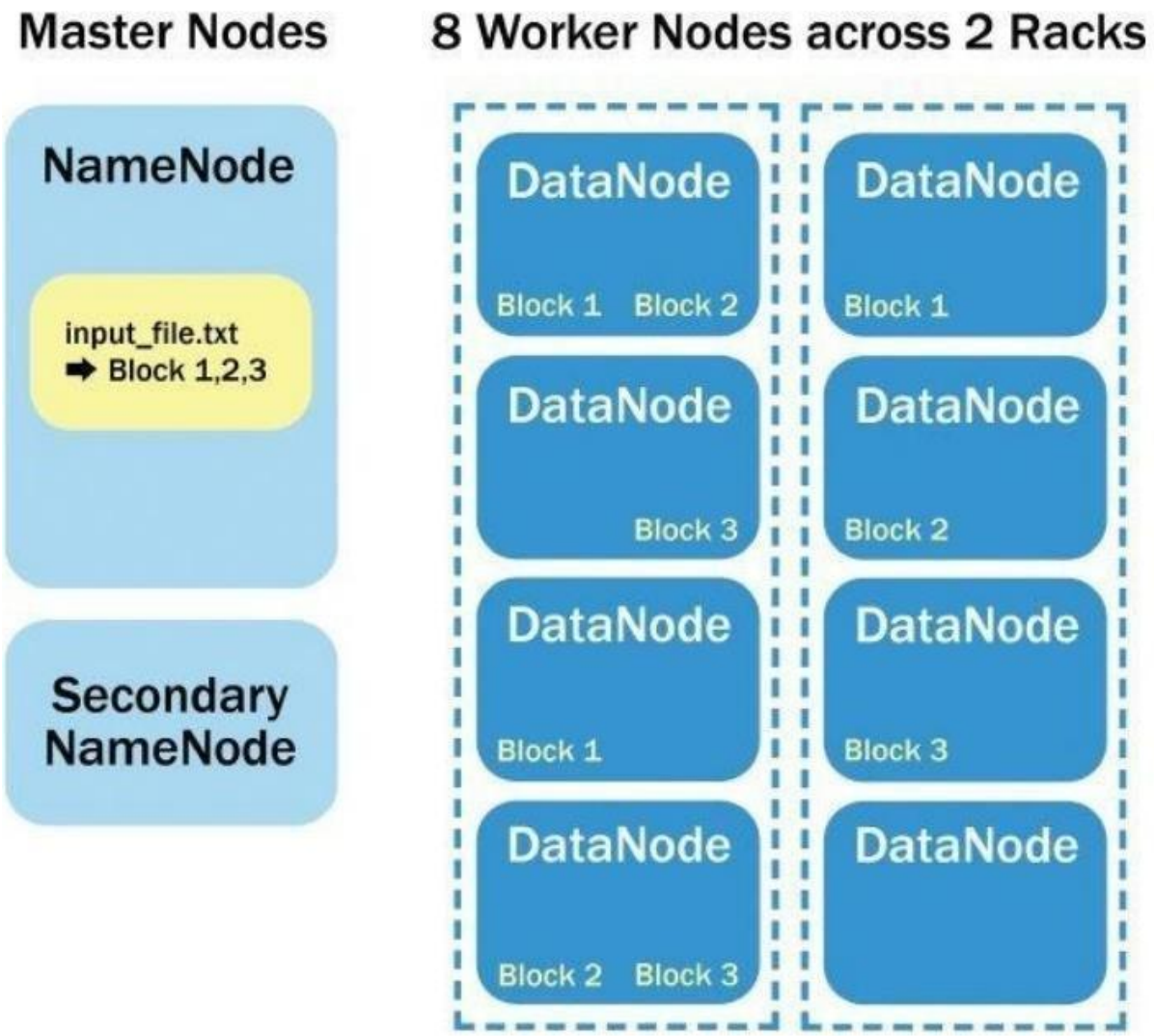


Figure 10.2 A file stored in HDFS Structuring a MapReduce Job in Hadoop Hadoop provides the ability to run MapReduce jobs as described, at a high level, in Section 10.1.2. This section offers specific details on how a MapReduce job is run in Hadoop. A typical MapReduce program in Java consists of three classes: the driver, the mapper, and the reducer. The driver provides details such as input file locations, the provisions for adding the input file to the map task, the names of the mapper and reducer Java classes, and the location of the reduce task output. Various job configuration options can also be specified in the driver. For example, the number of reducers can be manually specified in the driver. Such options are useful depending on how the MapReduce job output will be used in later downstream processing. The mapper provides the logic to be processed on each data block corresponding to the specified input files in the driver code. For example, in the word count MapReduce example provided earlier, a map task is instantiated on a worker node where a data block resides. Each map task processes a fragment of the text, line by line, parses a line into words, and emits for each word, regardless of how many times word appears in the line of text. The key/value pairs are stored temporarily in the worker node's memory (or cached to the node's disk). Next, the key/value pairs are processed by the built-in shuffle and sort functionality based on the number of reducers to be executed. In this simple example, there is only on

Practical No: 3

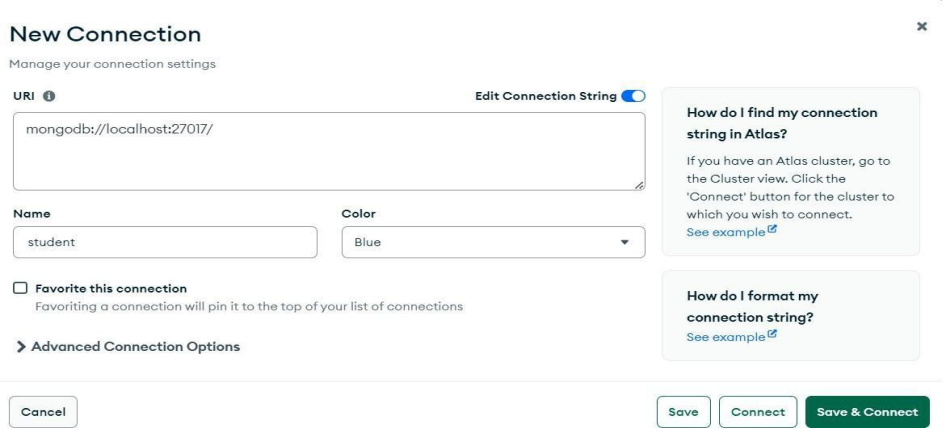
Aim: Implement an application that store big data in MongoDB

Description: MongoDB is a document oriented database is designed for NoSQL database to store and manage large amounts of unstructured or semi-structured data. Unlike traditional relational databases

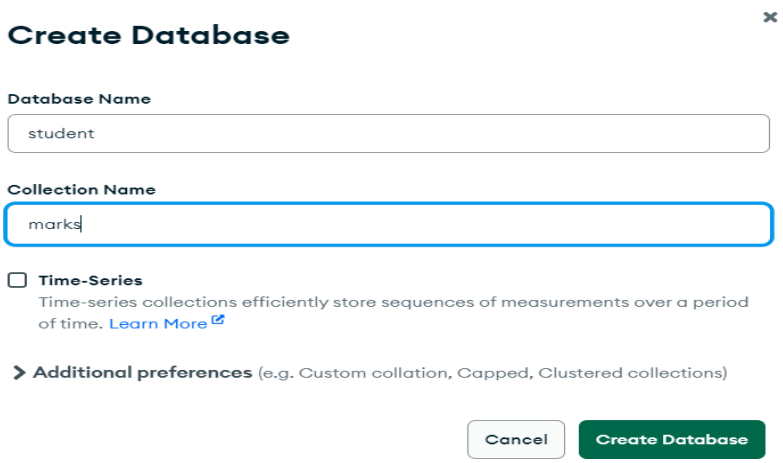
Step 1: Download MongoDB

Step 2: Setup to installation

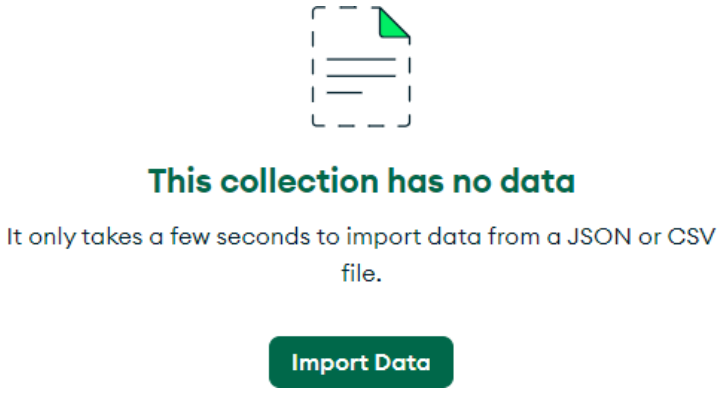
Step 3: Create a connection



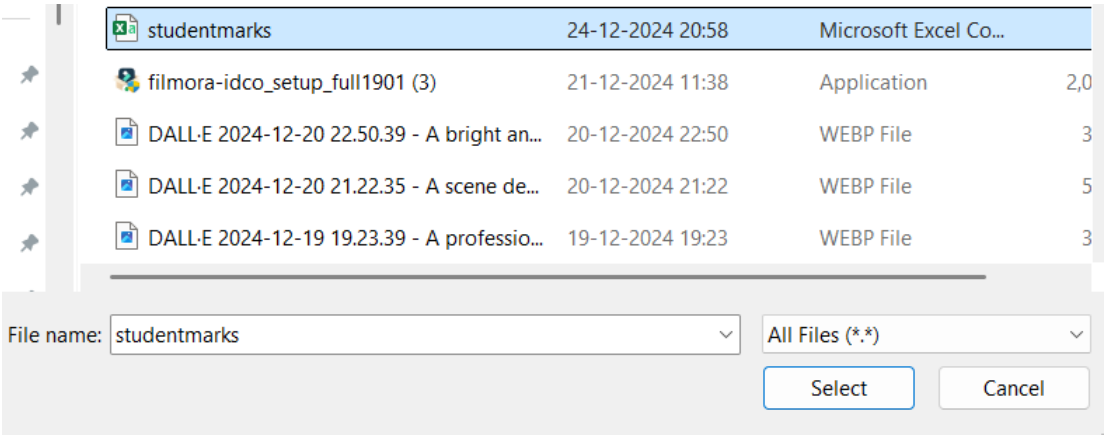
Step 4: Create a database and collection



Step 5: Import data



Step 6: Select the data



Step 7: Data is imported

Documents 400 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

ADD DATA | EXPORT DATA | UPDATE | DELETE

```
_id: ObjectId('677ffe466a35f8efd381264f')
admit : 0
gre : 380
gpa : 3.61
rank : 3
```

```
_id: ObjectId('677ffe466a35f8efd3812650')
admit : 1
gre : 660
gpa : 3.67
rank : 3
```

Step 8: Update the given data

```
_id: ObjectId('677ffe466a35f8efd381264f')
admit : 0
gre : 380
gpa : 3.61
rank : 3
```

```
1  _id: ObjectId('677ffe466a35f8efd381264f')
2  admit : 0
3  gre : 380
4  gpa : 9.1
5  rank : 3
```

Document modified.

Step 9: Performing deleting data

```
1  _id: ObjectId('677ffe466a35f8efd381264f')
2  admit : 0
3  gre : 380
4  gpa : 9.1
5  rank : 3
```

```
_id: ObjectId('677ffe466a35f8efd381264f')
admit : 0
gre : 380
gpa : 9.1
```

Step 10: Inserting data

```
_id: ObjectId('677ffe466a35f8efd381264f')
admit : 0
gre : 380
gpa : 9.1
```

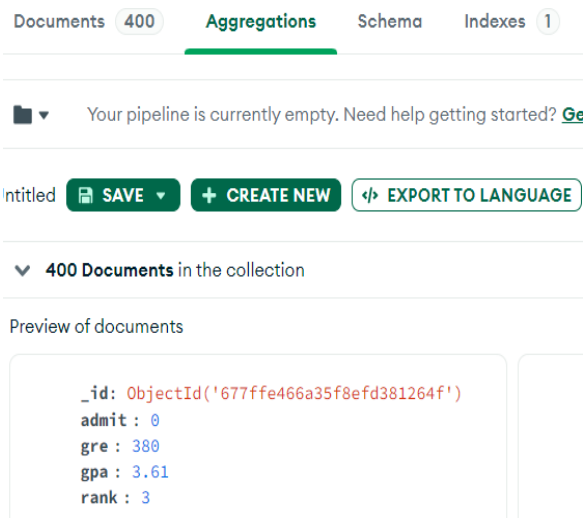
```
1  _id: ObjectId('677ffe466a35f8efd381264f')
2  admit : 0
3  gre : 380
4  gpa : 9.1
5  roll no : 52
```

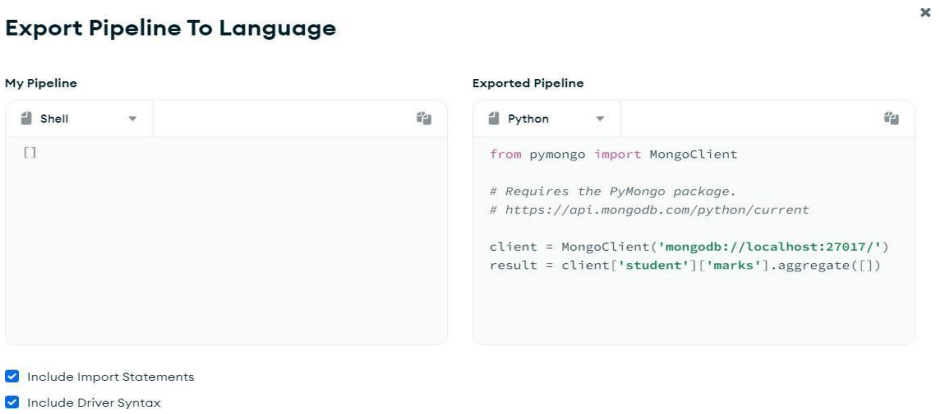
ObjectId
Int32
Int32
Double
Int32

Document modified. CANCEL UPDATE

```
_id: ObjectId('677ffe466a35f8efd381264f')
admit : 0
gre : 380
gpa : 9.1
roll no : 52
```

Step 11: Now go to aggregation and click export to language and select python





Step 12: Copy the code and paste in idle

```
from pymongo import MongoClient

# Requires the PyMongo package.
# https://api.mongodb.com/python/current

client = MongoClient('mongodb://localhost:27017/')
result = client['student']['marks'].aggregate([])
```

Step 13: Edit the code and Run the code and check the output

```
from pymongo import MongoClient

# Requires the PyMongo package.
# https://api.mongodb.com/python/current

client = MongoClient('mongodb://localhost:27017/')
result = client['student']['marks'].aggregate([])

print(list(client['student']['marks'].find()))
```

Click this two times

Squeezed text (474 lines).

Output

[{'_id': ObjectId('677ffe466a35f8efd381264f'), 'admit': 0, 'gre': 380, 'gpa': 9.1, 'roll no': 52}, {'_id': ObjectId('677ffe466a35f8
efd3812650'), 'admit': 1, 'gre': 660, 'gpa': 3.67, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd3812651'), 'admit': 1, 'gre': 800
, 'gpa': 4, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd3812652'), 'admit': 1, 'gre': 640, 'gpa': 3.19, 'rank': 4}, {'_id': Obje
ctId('677ffe466a35f8efd3812653'), 'admit': 0, 'gre': 520, 'gpa': 2.93, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd3812654'), 'a
dmit': 1, 'gre': 760, 'gpa': 3, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd3812655'), 'admit': 1, 'gre': 560, 'gpa': 2.98, 'ran
k': 1}, {'_id': ObjectId('677ffe466a35f8efd3812656'), 'admit': 0, 'gre': 400, 'gpa': 3.08, 'rank': 2}, {'_id': ObjectId('677ffe466a
35f8efd3812657'), 'admit': 1, 'gre': 540, 'gpa': 3.39, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd3812658'), 'admit': 0, 'gre':
700, 'gpa': 3.92, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd3812659'), 'admit': 0, 'gre': 800, 'gpa': 4, 'rank': 4}, {'_id': O
bjectId('677ffe466a35f8efd381265a'), 'admit': 0, 'gre': 440, 'gpa': 3.22, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd381265b'),
'admit': 1, 'gre': 760, 'gpa': 4, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd381265c'), 'admit': 0, 'gre': 700, 'gpa': 3.08, 'r
ank': 2}, {'_id': ObjectId('677ffe466a35f8efd381265d'), 'admit': 1, 'gre': 700, 'gpa': 4, 'rank': 1}, {'_id': ObjectId('677ffe466a3
5f8efd381265e'), 'admit': 0, 'gre': 480, 'gpa': 3.44, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd381265f'), 'admit': 0, 'gre':
780, 'gpa': 3.87, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd3812660'), 'admit': 0, 'gre': 360, 'gpa': 2.56, 'rank': 3}, {'_id'
: ObjectId('677ffe466a35f8efd3812661'), 'admit': 0, 'gre': 800, 'gpa': 3.75, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd3812662
'), 'admit': 1, 'gre': 540, 'gpa': 3.81, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd3812663'), 'admit': 0, 'gre': 500, 'gpa': 3
.17, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd3812664'), 'admit': 1, 'gre': 660, 'gpa': 3.63, 'rank': 2}, {'_id': ObjectId('6
77ffe466a35f8efd3812665'), 'admit': 0, 'gre': 600, 'gpa': 2.82, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd3812666'), 'admit':
0, 'gre': 680, 'gpa': 3.19, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd3812667'), 'admit': 1, 'gre': 760, 'gpa': 3.35, 'rank':
2}, {'_id': ObjectId('677ffe466a35f8efd3812668'), 'admit': 1, 'gre': 800, 'gpa': 3.66, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8
efd3812669'), 'admit': 1, 'gre': 620, 'gpa': 3.61, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd381266a'), 'admit': 1, 'gre': 520
, 'gpa': 3.74, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd381266b'), 'admit': 1, 'gre': 780, 'gpa': 3.22, 'rank': 2}, {'_id': O
bjectId('677ffe466a35f8efd381266c'), 'admit': 0, 'gre': 520, 'gpa': 3.29, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd381266d'),
'admit': 0, 'gre': 540, 'gpa': 3.78, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd381266e'), 'admit': 0, 'gre': 760, 'gpa': 3.35,
'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd381266f'), 'admit': 0, 'gre': 600, 'gpa': 3.4, 'rank': 3}, {'_id': ObjectId('677ffe4
66a35f8efd3812670'), 'admit': 1, 'gre': 800, 'gpa': 4, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd3812671'), 'admit': 0, 'gre':
360, 'gpa': 3.14, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd3812672'), 'admit': 0, 'gre': 400, 'gpa': 3.05, 'rank': 2}, {'_id'
: ObjectId('677ffe466a35f8efd3812673'), 'admit': 0, 'gre': 580, 'gpa': 3.25, 'rank': 1}, {'_id': ObjectId('677ffe466a35f8efd3812674
'), 'admit': 0, 'gre': 520, 'gpa': 2.9, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd3812675'), 'admit': 1, 'gre': 500, 'gpa': 3.
13, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd3812676'), 'admit': 1, 'gre': 520, 'gpa': 2.68, 'rank': 3}, {'_id': ObjectId('67
7ffe466a35f8efd3812677'), 'admit': 0, 'gre': 560, 'gpa': 2.42, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd3812678'), 'admit': 1
, 'gre': 580, 'gpa': 3.32, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd3812679'), 'admit': 1, 'gre': 600, 'gpa': 3.15, 'rank': 2
}, {'_id': ObjectId('677ffe466a35f8efd381267a'), 'admit': 0, 'gre': 500, 'gpa': 3.31, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8e
fd381267b'), 'admit': 0, 'gre': 700, 'gpa': 2.94, 'rank': 2}, {'_id': ObjectId('677ffe466a35f8efd381267c'), 'admit': 1, 'gre': 460,
'gpa': 3.45, 'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd381267d'), 'admit': 1, 'gre': 580, 'gpa': 3.46, 'rank': 2}, {'_id': Obj
ectId('677ffe466a35f8efd381267e'), 'admit': 0, 'gre': 500, 'gpa': 2.97, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd381267f'), '
admit': 0, 'gre': 440, 'gpa': 2.48, 'rank': 4}, {'_id': ObjectId('677ffe466a35f8efd3812680'), 'admit': 0, 'gre': 400, 'gpa': 3.35,
'rank': 3}, {'_id': ObjectId('677ffe466a35f8efd3812681'), 'admit': 0, 'gre': 640, 'gpa': 3.86, 'rank': 3}, {'_id': ObjectId('677ffe
466a35f8efd3812682'), 'admit': 0, 'gre': 440, 'gpa': 3.13, 'rank': 4}, {' id': ObjectId('677ffe466a35f8efd3812683'), 'admit': 0, 'g

Practical No: 4

Aim: Implement a program in Pig

Description: Apache Pig is a high-level platform for processing large datasets in Hadoop. It provides a scripting language called **Pig Latin**, which is used to write data transformation scripts that are then converted into MapReduce jobs or executed using Apache Tez or Apache Spark.

What is Apache Pig Architecture?

Apache Pig is a **high-level data processing platform** built on **Hadoop**. It simplifies **big data processing** by using **Pig Latin**, a data flow language that allows efficient handling of **large datasets** in a **distributed and parallel** manner.

Apache Pig Architecture and Components

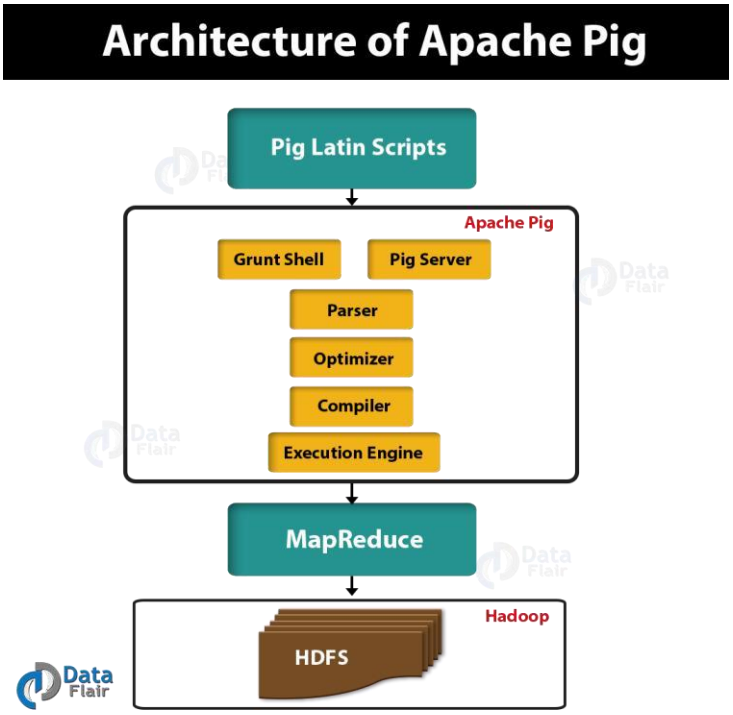
1. Data Storage (Hadoop HDFS)

- **Raw data** comes from multiple sources: IoT devices, websites, mobile applications, logs, etc.
- Stored in **Hadoop Distributed File System (HDFS)** for processing.

2. Pig Framework

- **Converts Pig Latin scripts into MapReduce jobs** for parallel execution on Hadoop.
- Acts as a **middleware** between **HDFS and execution engines** like **MapReduce, Tez, or Spark**.

3. Pig Components and Execution Flow



Architecture of Apache Pig

Apache Pig Components

There are several components in the Apache Pig framework. Let's study these major components in detail:

1. Parser

- The **first step** in Pig execution.
- Handles **Pig Latin scripts** and commands in the **Grunt shell**.
- **Performs checks**: Syntax validation Type checking
  Other script validations
- **Generates a Directed Acyclic Graph (DAG)**: **Nodes** = Logical operators
  Edges = Data flow

2. Optimizer

- Receives the **DAG from the parser** and optimizes it.
- Uses **projection and pushdown** to: Remove unnecessary columns
  Reduce data loading, improving query performance

3. Compiler

- Converts the **optimized logical plan** into **MapReduce jobs**.
- **Automates optimization**, so the programmer **does not** need to manually tune the script.
- **Reorders execution sequences** to maximize performance while maintaining correctness.

4. Execution Engine

- Submits the **compiled MapReduce jobs** to **Hadoop**.
- The **MapReduce framework** executes these jobs to process data.

5. Execution Modes in Pig

Pig supports **two execution modes** based on data availability and where the script runs.

1. Local Mode

- Best suited for **small datasets**.
- **Runs on a single JVM** (Java Virtual Machine).
- **No parallel processing**, as execution is done locally.
- Pig loads data from the **local file system**.

Command to start Pig in Local Mode:

pig -x local

To run Pig in **Tez Local Mode** (using Tez runtime):

pig -x tez_local

2. MapReduce Mode (MR Mode)

- Requires **Hadoop cluster** and **HDFS setup**.
- Processes data stored in **HDFS**.
- By default, Pig runs in **MapReduce mode**, so no need to specify it explicitly.

Command to start Pig in MR Mode:

pig -x mapreduce

3. Pig Job Execution Flow

Step-by-step process of execution in Pig:

- **Programmer writes a Pig Latin script** and stores it in the local file system.
- **The script is submitted to the Pig Compiler**, which converts it into **MapReduce jobs**. **Pig retrieves raw data from HDFS** and executes the script.
- **MapReduce jobs process the data**, applying transformations and filters.
- **The final result is stored back in HDFS** after execution.

Practical No: 5

Aim: Illustrate the working of JAQL

Description: JAQL (JSON Query Language) is a high-level, declarative language specifically designed for processing and analyzing JSON (JavaScript Object Notation) data. Originally developed by IBM, it is widely used in big data and Hadoop environments for managing semi-structured and unstructured data efficiently.

Working of JAQL

JAQL follows a functional programming paradigm and processes JSON data through a series of transformations and operations.

1. JSON Data Representation

JAQL is designed to work with JSON, which stores data in a flexible key-value format.

Example:

jaql> a = { name : "Scott", age : 42, children : ["Jake", "Sam"]

}; jaql> a.name;

"Scott"

jaql>

a.children[0];

"Jake"

2. Input/Output Operations in JAQL

JAQL uses I/O adapters to read and write data across different sources like local files, HDFS, and URLs.

Examples:

- Writing data to a CSV file:

  jaql> [1, 2, 3, 4, 5] -> write(del("test.csv"));

  jaql> read(del("test.csv"));

  [1, 2, 3, 4, 5]

- Reading from Local Files and HDFS:

  jaql> read(del("file:///home/user/test.csv")); // Local File

  jaql> read(del("hdfs://localhost:9000/user/test.csv")); // HDFS File

- Reading JSON Data from a URL:

  jaql> jaqlGet("file:///tmp/test.txt");

3. Data Manipulation (Core Operators)

JAQL provides a set of operators to transform and analyze data, similar to SQL.

- Filtering Data

The filter operator is used to select specific elements based on conditions.

```
jaql> data = [1, 2, 3, 4, 5, 6, 7, 8, 9];
jaql> data -> filter each num (3 <= num <=
6); [ 3, 4, 5, 6 ]
```

- Transforming Data

The transform operator applies an expression to each element in an array.

```
jaql> recs = [ {a: 1, b: 4}, {a: 2, b: 5}, {a: -1, b: 4} ];
jaql> recs -> transform { sum: $.a + $.b
}; [ { sum: 5 }, { sum: 7 }, { sum: 3 } ]
```

- Sorting Data

The sort operator orders records based on a specified field.

```
jaql> read(del("file:///path/to/people.txt")) -> sort by [$.age asc];
```

- Grouping Data

The group by operator categorizes data based on a key.

```
jaql> employees -> group by $.department into { "Department": key, "Employees": $ };
```

4. Execution on Hadoop

JAQL supports distributed processing using Hadoop MapReduce, making it ideal for large-scale data processing.

Queries are executed in parallel across clusters for better performance.

5. Writing Processed Data

JAQL allows saving transformed data back into different storage formats like JSON, CSV, and HDFS.

Example:

```
write(employees -> filter $.salary > 6000, hdfs("high_salary_employees.json"));.
```

Practical No: 6

Aim: Implement Decision Tree Classification technique

Description: A **Decision Tree** is a **supervised learning algorithm** used for **classification and regression** tasks. It works like a **flowchart**, where decisions are made based on **conditions** at each node.

Code

Decision Tree

Classification # Importing

the dataset

Step 1: Importing the Dataset

dataset = read.csv("C:\\Users\\ADMIN\\Downloads\\socialnetworking.csv")

#print(dataset)

dataset = dataset[3:5] # columns 3 4 ad 5

print(dataset)

Encoding the target feature as factor(just like a vector having levels

levels to convey that only two possible values for purchased - 0 & 1

Step 2: Converting the PurchasedColumn into Categories

dataset$Purchased = factor(dataset$Purchased, levels = c(0, 1)) print

(dataset$Purchased)

Splitting the dataset into the Training set and Test set

Step 3: Splitting the Dataset into Training and Test

Sets install.packages('caTools')

library(caTools

) set.seed(123)

#split = sample.split(dataset$Purchased, SplitRatio = 0.75)

split = sample.split(dataset$Purchased, SplitRatio = 0.75)

training_set = subset(dataset, split == TRUE)

test_set = subset(dataset, split == FALSE)

Feature Scaling - scale() method centers and/or scales the #columns of a numeric matrix.

Step 4: Feature Scaling

```
training_set[-3] = scale(training_set[-3])
# scaling first 2 columns, don't consider 3rd column
test_set[-3] = scale(test_set[-3])
#print(test_set[-3])
# Fitting Decision Tree Classification to the Training set
```

Step 5: Training the Decision Tree Classifier

```
install.packages('rpart')
library(rpart) # for partitioning tree
install.packages('rpart.plot')
library(rpart.plot)
classifier = rpart(formula = Purchased ~ .,data = training_set)
# Predicting the Test set results
```

Step 6: Predicting the Test Set Results

```
y_pred = predict(classifier, newdata = test_set[-3], type = 'class')
print(y_pred)
# Making the Confusion Matrix
```

Step 7: Evaluating the Model using a Confusion Matrix

```
cm = table(test_set[, 3], y_pred)
print(cm)
# Plotting the tree
#extra=106 class model with a binary response
#extra=104 class model with a response having more than two #levels
```

Step 8: Visualizing the Decision Tree

```
rpart.plot(classifier, extra = 106)
```

OUTPUT

```
# Predicting the Test set results
print(y_pred)
2  4  5  9  12  18  19  20  22  29  32  34  35  38  45  46  48  52  66  69  74
0  0  0  0  0  0  1  1  0  0  1  0  1  0  0  0  0  0  0  0  1
```

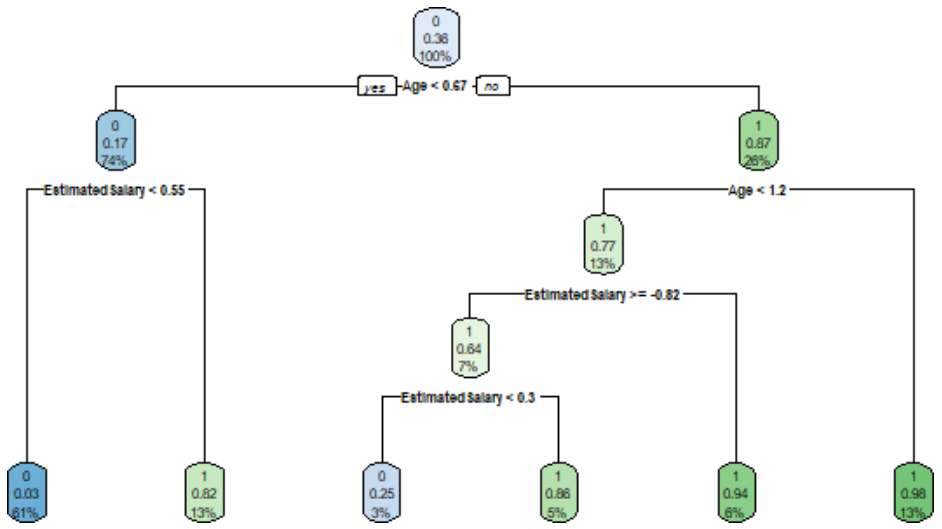
```
75  82  84  85  86  87  89 103 104 107 108 109 117 124 126 127 131 134 139 148 154
 0   0   1   0   1   0   0   1   1   0   1   1   0   0   0   0   0   0   0   0   0
156 159 162 163 170 175 176 193 199 200 208 213 224 226 228 229 230 234 236 237 239
  0   0   1   0   0   0   0   0   0   0   1   1   1   0   1   0   0   1   1   0   1
Levels: 0 1
```

```
# Making the Confusion Matrix
> cm = table(test_set[, 3], y_pred)
> print(cm)
   y_pred
     0  1
  0 53 11
  1  6 30
```

Plotting the tree

#extra=106 class model with a binary response

#extra=104 class model with a response having more than two #levels



Practical No: 7

Aim: Implement Support Vector Machine (SVM) Classification techniques

Description: SVM is a **supervised machine learning algorithm** used for **classification and regression** tasks. It helps **separate data into different categories** using a special boundary called a **hyperplane**.

Code

Support Vector Machine

(SVM) # Importing the

dataset

Step 1: Importing the Dataset

dataset =

read.csv("C:\\Users\\dhruv\\Downloads\\socialnetworking.csv")

dataset = dataset[3:5]

print(dataset)

print(dataset$Purchase

d)

Splitting the dataset into the Training set and Test

set **Step 2: Splitting the Dataset into Training and**

Test Sets install.packages('caTools')

library(caTool

s)

set.seed(123)

split = sample.split(dataset$Purchased, SplitRatio =

0.75) training_set = subset(dataset, split == TRUE)

print(training_set)

test_set = subset(dataset, split == FALSE)

print(test_set)

Feature Scaling

Step 3: Feature Scaling

training_set[-3] = scale(training_set[-3]) # [-3] means 3rd index will be

dropped test_set[-3] = scale(test_set[-3])

```
print(training_set[-
3]) print (test_set[-
3])
# Fitting SVM to the Training set
```

Step 4: Fitting the SVM Model to the Training Set

```
install.packages('e107
1') library(e1071)
classifier = svm(formula = Purchased ~
        ., data = training_set,
        type = 'C-
        classification', kernel
        = 'linear')
        print (classifier)
```

Step 5: Predicting the Test Set Results

```
y_pred = predict(classifier, newdata = test_set[-3])
print(y_pred)
```

Step 6: Making the Confusion Matrix

```
cm = table(test_set[, 3], y_pred)
print
(cm)
```

Output

```
215  47        43000       0
216  60       108000       1
217  49        65000       0
218  40        78000       0
219  46        96000       0
220  59       143000       1
221  41        80000       0
222  35        91000       1
223  37       144000       1
```

```
[ reached 'max' / getOption("max.print") -- omitted 67 rows ]
  >print(dataset$Purchased)

 [191] 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 1 1 1 0 0 1 1

 [229] 0 1 1 0 1 1 0 1 0 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 1 1 0 1 0 0 1 1 0 1 1 0 1 1

 > # Splitting the dataset into the Training set and Test set

 >  >instll.packages('caTools')

 Error in install.packages : Updating loaded packages

 > library(caTools)

 > set.seed(123)

 > split = sample.split(dataset$Purchased, SplitRatio = 0.75)

 > training_set = subset(dataset, split == TRUE)

 > print(training_set)

    Age EstimatedSalary Purchased

 8   32         150000         1

 10  35          65000         0

 11  26          80000         0

 13  20          86000         0

 14  32          18000         0

 > # Feature Scaling

 > training_set[-3] = scale(training_set[-3]) # [-3] means 3rd index will be dropped

 > test_set[-3] = scale(test_set[-3])

 > Age EstimatedSalary

 1   -1.76554750     -1.47334137

 3   -1.09629664     -0.78837605

 6   -1.00068938     -0.36027273

 > print (test_set[-3])

          Age EstimatedSalary

 2   -0.30419063     -1.51354339

 4   -1.05994374     -0.32456026

 5   -1.81569686      0.28599864
```

```
> # Fitting SVM to the Training set
> install.packages('e1071')
Error in install.packages : Updating loaded packages
> library(e1071)
> classifier = svm(formula = Purchased ~ .,
+                  data = training_set,
+                  type = 'C-classification',
+                  kernel = 'linear')
> print
(classifier) Call:
svm(formula = Purchased ~ ., data = training_set, type = "C-
    classification", kernel = "linear")
Parameters:
   SVM-Type: C-
 classification SVM-Kernel:
 linear
       cost:  1
Number of Support Vectors:  116
> # Predicting the Test set results
> y_pred = predict(classifier, newdata = test_set[-3])
> print(y_pred)
 74  75  82  84  85  86  87  89 103 104 107 108 109 117 124 126 127 131 134 139
  0   0   0   0   0   0   0   0   0   1   0   0   0   0   0   0   0   0   0   0
Levels: 0 1
> # Making the Confusion Matrix
> cm = table(test_set[, 3], y_pred)
> print (cm)
   y_pred
    0  1
  0 57  7
```

Practical No: 8

Aim: Regression Model Import a data from web storage. Perform Logistic Regression to find out relation between variables that are affecting the admission of a student in an institute based on his or her GRE score, GPA obtained and rank of the student. Also check the model is fit or not require (foreign), require (Mass)

Description: A **regression model** is used to find the relationship between variables. It helps us understand how one or more **independent variables (inputs)** affect a **dependent variable (output).**

Logistic Regression : Logistic Regression (For Classification Problems) Used when the dependent variable is categorical (e.g., Yes/No, Admit/Not Admit). Predicts the probability of an outcome.

Logistic Regression (Predicting Student Admission)

Code

Step 1: Load the Dataset

```
college <- read.csv("C:\\Users\\ADMIN\\Downloads\\studentmarks.csv")

head(college)

nrow(college)
```

Step 2: Split the Dataset

```
install.packages("caTools")   # For Logistic

regression library(caTools)

split <- sample.split(college, SplitRatio =

0.75) split

training_reg <- subset(college, split ==

"TRUE") test_reg <- subset(college, split ==

"FALSE")

# Training model
```

Step 3: Train Logistic Regression Model

```
fit_logistic_model <- glm(admit ~ .,

                data =

                training_reg,

                family =

                "binomial"
```

Step 4: Make Predictions on Test Data

```
predict_reg <- predict(fit_logistic_model,
                       test_reg, type = "response")
predict_reg
cdplot(as.factor(admit)~ gpa,
data=college)  cdplot(as.factor(admit)~
gre, data=college)
cdplot(as.factor(admit)~ rank,
data=college) # Changing probabilities
```

Step 5: Convert Probabilities into Binary Classifications

```
predict_reg <- ifelse(predict_reg >0.5, 1, 0)
predict_reg
accuracy # using
confusion matrix
```

Step 6: Evaluate Model Accuracy (Confusion Matrix)

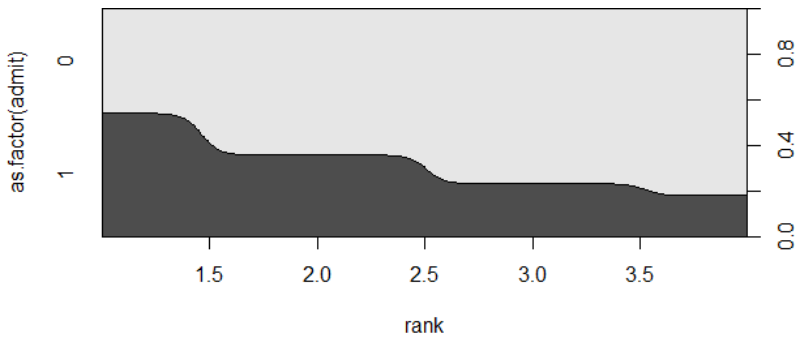
```
table(test_reg$admit, predict_reg)
```

Output

```
#fetch the data
> college <- read.csv("C:\\Users\\ADMIN\\Downloads\\studentmarks.csv")
> head(college)
  admit gre  gpa rank
1     0 380 3.61    3
2     1 660 3.67    3
3     1 800 4.00    1
4     1 640 3.19    4
5     0 520 2.93    4
6     1 760 3.00    2
> nrow(college)
[1] 400
```

```
Error in install.packages : Updating loaded packages
> library(caTools)
> split <- sample.split(college, SplitRatio = 0.75)
> split
[1] FALSE  TRUE  TRUE  TRUE
> training_reg <- subset(college, split == "TRUE")
> test_reg <- subset(college, split == "FALSE")
> # Training model
> fit_logistic_model <- glm(admit ~ .,
+                           data = training_reg,
+                           family = "binomial")
> # Predict test data based on model
> predict_reg <- predict(fit_logistic_model,
+                        test_reg, type = "response")
> predict_reg
        1         5         9        13        17        21        25
0.17418571 0.12118847 0.23609659 0.69706184 0.35280630 0.19318446 0.49492217
       29        33        37        41        45        49        53
0.49197706 0.27344718 0.45614574 0.23186479 0.38944282 0.07603148 0.26412898
       57        61        65        69        73        77        81
0.22676947 0.36235289 0.33349616 0.51991716 0.13687566 0.24454949 0.19386608
       85        89        93        97       101       105       109
0.23511001 0.55575097 0.60506675 0.22163974 0.12445479 0.50238935 0.13152920
      113       117       121       125       129       133       137
0.12192734 0.27251485 0.36388731 0.42164835 0.30697994 0.36242926 0.16500928
> cdplot(as.factor(admit)~ gpa, data=college)
> cdplot(as.factor(admit)~ gre, data=college)
> cdplot(as.factor(admit)~ rank, data=college)
> # Changing probabilities
> predict_reg <- ifelse(predict_reg >0.5, 1, 0)
```

```
> predict_reg
  1   5   9  13  17  21  25  29  33  37  41  45  49  53  57  61  65  69  73  77  81
  0   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0   0   0
 85  89  93  97 101 105 109 113 117 121 125 129 133 137 141 145 149 153 157 161 165
  0   1   1   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
169 173 177 181 185 189 193 197 201 205 209 213 217 221 225 229 233 237 241 245 249
  0   0   0   0   0   0   0   0   0   1   0   0   0   0   0   0   0   0   0   0   0
> # Evaluating model accuracy
> # using confusion matrix
> table(test_reg$admit, predict_reg)
   predict_reg
    0  1
  0 69  5
  1 22  4
```



Practical No: 9

Aim: MULTIPLE REGRESSION MODEL: Apply multiple regressions, if data have a continuous independent variable. Apply on above dataset.

Description: Multiple Regression (For Continuous Data) Used when there are multiple independent variables affecting a dependent variable.

Code

Step 1: Load the Dataset

```
studentData<-
read.csv("C:\\Users\\ADMIN\\Downloads\\studentmarks.csv")
head(studentData)
nrow(studentData
) #logistic
regression
```

Step 2: Split the Dataset

```
library(caTools)
Split<-sample.split(studentData,SplitRatio = 0.75)
Split
training_reg<-subset(studentData,split ==
"TRUE") training_reg<-
subset(studentData,split == "FALSE")
```

Step 3: Train Multiple Regression Model

```
fit_MRegressor_model<- lm(formula = admit ~ gre+gpa+rank, data = training_reg)
```

Step 4: Make Predictions on Test Data

```
predict_reg<-predict(fit_MRegressor_model,newdata =
test_reg) predict_reg
```

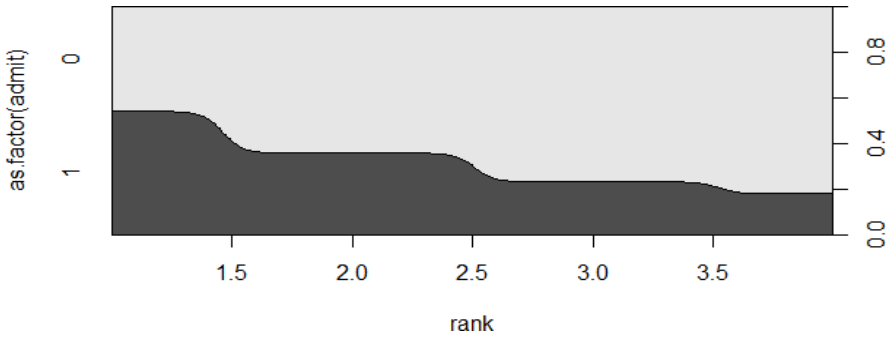
Step 5: Visualize Regression Relationships

```
cdplot(as.factor(admit)~ gpa,data = studentData)
cdplot(as.factor(admit)~ gre,data = studentData)
cdplot(as.factor(admit)~ rank,data = studentData)
```

Output

```
studentData<-read.csv("C:\\Users\\ADMIN\\Downloads\\studentmarks.csv")
```

```
> head(studentData)
  admit gre  gpa rank
1     0 380 3.61    3
2     1 660 3.67    3
> nrow(studentData)
[1] 400
> #logistic regression
> library(caTools)
> Split<-sample.split(studentData,SplitRatio = 0.75)
> Split
[1]  TRUE FALSE  TRUE  TRUE
> training_reg<-subset(studentData,split == "TRUE")
> training_reg<-subset(studentData,split == "FALSE")
> fit_MRegressor_model<- lm(formula = admit ~ gre+gpa+rank, data = training_reg)
> predict_reg<-predict(fit_MRegressor_model,newdata = test_reg)
> predict_reg
          1           5           9          13          17          21
0.241855267 -0.055830475 0.186674004 0.597142772 0.113195917 0.145639736
         25          29          33          37          41          45
0.315341336 0.288007197 0.184423708 0.460339840 0.144064649 0.237839510
         49          53          57          61          65          69
> cdplot(as.factor(admit)~ gpa,data = studentData)
> cdplot(as.factor(admit)~ gre,data = studentData)
> cdplot(as.factor(admit)~ rank,data = studentData)
```



Practical No: 10

Aim: Classification Model:

a) Install relevant packages
b) choose a classifier, and
c) evaluate its performance.

Description : A **classification model** is a type of machine learning model that categorizes input data into predefined classes or labels based on its learned patterns. It is a supervised learning approach where the model is trained on labeled data and then used to predict the category of new, unseen data.

Naive Bayes: Naive Bayes is part of a family of generative learning algorithms, meaning that it seeks to model the distribution of inputs of a given class or category. Unlike discriminative classifiers, like logistic regression, it does not learn which features are most important to differentiate between classes.

Step 1:- importing the

dataset. Iris

Step 2:- loading packages:

 # Loading

package

library(e1071)

library(caTools)

library(caret)

step 2:-Splitting data into train and test

data split <- sample.split(iris, SplitRatio

= 0.7) train_cl <- subset(iris, split ==

"TRUE") test_cl <- subset(iris, split ==

"FALSE")

step 3:- scaling the data set

train_scale <- scale(train_cl[,

1:4]) test_scale <-

scale(test_cl[, 1:4])

step 4:- Fitting Naive Bayes Model to training dataset

set.seed(120) # Setting Seed

classifier_cl <- naiveBayes(Species ~ ., data = train_cl)

classifier_cl

```
A-priori probabilities:
Y
     setosa versicolor  virginica
 0.3333333  0.3333333  0.3333333

Conditional probabilities:
            Sepal.Length
Y                [,1]      [,2]
  setosa      4.973333 0.3084257
  versicolor  5.966667 0.4929386
  virginica   6.520000 0.6764002

            Sepal.Width
Y                [,1]      [,2]
  setosa      3.426667 0.3561609
  versicolor  2.776667 0.2712466
  virginica   2.976667 0.3607304

            Petal.Length
Y                [,1]      [,2]
  setosa      1.453333 0.1775957
  versicolor  4.243333 0.4328600
  virginica   5.496667 0.5505379

            Petal.Width
Y                [,1]        [,2]
  setosa      0.2333333 0.09222661
  versicolor  1.3233333 0.19419743
  virginica   1.9900000 0.27586853
```

Step 5:- Predicting on test data

y_pred <- predict(classifier_cl, newdata = test_cl)

step 6 :- confusion matrix

cm <- table(test_cl$Species, y_pred)

> cm

```
            y_pred
             setosa versicolor virginica
  setosa         20          0         0
  versicolor      0         19         1
  virginica       0          1        19
```

Step 7 :- confusion matrix

confusionMatrix(cm

```
Confusion Matrix and Statistics

            y_pred
             setosa versicolor virginica
  setosa         20          0         0
  versicolor      0         19         1
  virginica       0          1        19

Overall Statistics

               Accuracy : 0.9667
                 95% CI : (0.8847, 0.9959)
    No Information Rate : 0.3333
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.95

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor
Sensitivity                 1.0000            0.9500
Specificity                 1.0000            0.9750
Pos Pred Value              1.0000            0.9500
Neg Pred Value              1.0000            0.9750
Prevalence                  0.3333            0.3333
Detection Rate              0.3333            0.3167
Detection Prevalence        0.3333            0.3333
Balanced Accuracy           1.0000            0.9625
                     Class: virginica
```

```
                     Class: virginica
Sensitivity                    0.9500
Specificity                    0.9750
Pos Pred Value                 0.9500
Neg Pred Value                 0.9750
Prevalence                     0.3333
Detection Rate                 0.3167
Detection Prevalence           0.3333
Balanced Accuracy              0.9625
>
```

Practical No: 11

Aim: A. Clustering algorithms for unsupervised classification

Description: Clustering algorithms are unsupervised machine learning techniques used to group data points based on their similarity. Unlike supervised classification, clustering does not use labeled data; instead, it discovers inherent patterns within the dataset.

Code

Step 1: Import the Dataset

```
dataset=read.csv("C:\\Users\\ADMIN\\Downloads\\Mall_Customers.csv")

head(dataset)

dataset=dataset[4:5]

head(dataset)
```

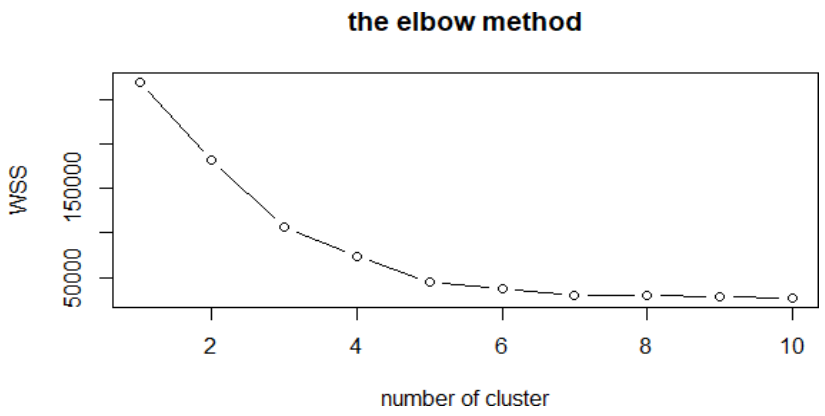
Step 2: Compute Within-Cluster Sum of Squares (WCSS)

```
wcss=vector()

for (i in 1:10)wcss[i]=sum(kmeans(dataset,i)$withinss)
```

Step 3: Use the Elbow Method to Find Optimal Clusters

```
plot(1:10,

wcss, type ='b',

main = paste('the elbow method'), xlab =

'number of cluster',

ylab = 'WSS')
```

Output



Aim: B. Plot the cluster data using R visualizations

Description: K-Means Clustering : K-Means Clustering is an unsupervised machine learning algorithm used for clustering or grouping data points into a predefined number of clusters (denoted as K). It's a simple and widely-used technique in data analysis, especially for segmentation or grouping similar data points.

code

Step 1: Apply K-Means Clustering kmeans

= kmeans(x = dataset,centers =5)

y_kmeans=kmeans$cluster

Step 2: Visualize the Clusters

library(cluster) clusplot(dataset,

y_kmeans, lines =0,

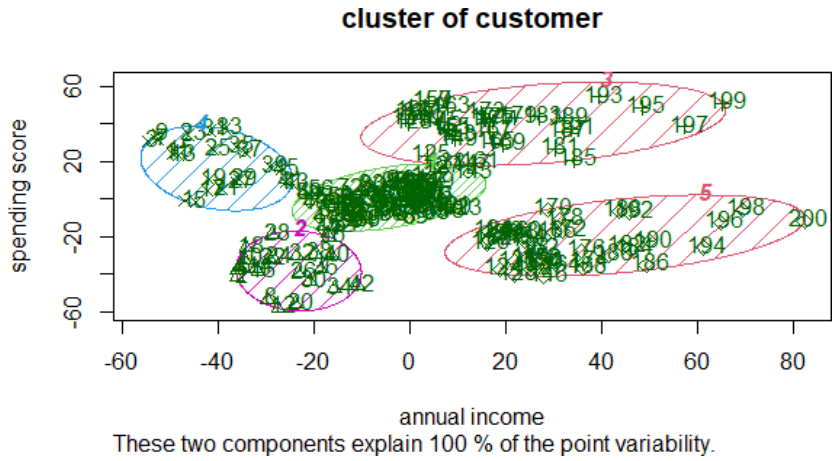
shade = TRUE,

color = TRUE,

labels = 2,

main = paste('cluster of customer'), xlab

= 'annual income',

ylab = 'spending score')

Output